ParseAnything — The Trust Layer for Document AI

DataQuest 3.0 · DQCL · Complete Winning Execution Blueprint

> **Tagline:** *Every page parsed. Every number traceable. Nothing hallucinated.*

---

0. TL;DR

Most teams will build "PDF → Markdown with OCR." That is the baseline the problem statement says we must *decisively outperform*.

We build **ParseAnything**: one entry point that **detects → routes → extracts → verifies → flags**, and — the part nobody else will ship — a **Provenance Viewer** where every output block is clickable back to its exact bounding box on the source page, color-coded by confidence, with **self-verification** (financial tables prove their own totals, charts re-plot themselves over the original to prove the extracted numbers).

Three promises that map 1:1 to the judging rubric:

Judge criterion (from problem statement)	Our answer
Benchmark accuracy (text, tables, charts, math, reading order, bbox provenance)	Specialist extractor per region + cross-validation + live **Benchmark Arena** with real metrics
Production practicality (throughput, <$10/1k pages, self-hosted, failure handling)	**Cost-aware escalation ladder** (free CPU first, VLM only when needed), Docker self-host, license-clean stack, <60s structured errors
Trust in high-stakes domains	**Flag, don't hallucinate**: per-block confidence, numeric integrity checks, human review queue

---

1. The BEST Enhanced Solution Idea

Concept: "Detect → Route → Extract → Verify → Fail-Safe"

The problem statement defines 3 stages (Detect & Route, Extract & Assemble, Flag & Fail Safe). We add a **4th, differentiating stage — Verify** — and make all four *visible* in the UI.

```
FILE ──► [1 DETECT & ROUTE] ──► [2 EXTRACT & ASSEMBLE] ──► [3
VERIFY] ──► [4 FLAG & FAIL-SAFE] ──► MD + JSON + Viewer
          format sniff           typed blocks, reading
sum-checks,       confidence per block,
          per-page/region        order, cross-page tables,
chart re-plot,    structured errors <60s,
          classification         bboxes
cross-extractor   never invent text

agreement
```

The core product insight

Finance/legal AI doesn't fail because OCR is 2% worse. It fails because **nobody can tell which 2% is wrong**. So our product is not just a parser — it is a **parser that knows what it doesn't know** and lets a human verify any claim in one click.

The "escalation ladder" (this is what makes <$10/1k pages real)

Level	Method	Cost	Used for
L0	Native text layer / native Office XML / native cell grid	~free, ms	Digital PDFs, DOCX, XLSX
L1	Classical CV + CPU OCR (RapidOCR) + rule-based table/layout	~free	Scans, simple regions
L2	Specialist local models (layout, table-structure, LaTeX OCR)	~free (CPU/GPU)	Complex tables, equations

Level	Method	Cost	Used for
L3	Vision-LLM (Gemini free tier **or** local Qwen2.5-VL via Ollama)	small $	Charts, low-confidence regions, self-heal

A region climbs the ladder **only if its confidence stays below threshold**. Typical corpus: ~85–90% of regions never leave L0–L1. *(Measure this in the benchmark and show the real number — don't quote ours until you've measured.)*

---

2. Why This Solution Will Win

1. **It answers the rubric literally.** Every bullet in the problem statement (invisible content, broken tables, lost figures/math, scrambled order, format sprawl, no provenance) is a visible demo moment.
2. **It's the only one that is *auditable*.** Click a number → see the pixels it came from.
3. **It treats cost and failure as features**, not afterthoughts (live cost meter, <60s structured errors, partial results).
4. **It looks like a product**, not a script: polished viewer, benchmark arena, API docs, Docker one-liner.
5. **It's honest.** "Flag instead of hallucinate" is *the* sentence in the brief that most teams will ignore. We make it the headline.
6. **Startup-grade:** clear buyer (Cenizas Labs-style vertical AI vendors), clear pricing story, clear moat (verification + provenance data).

---

3. Unique Differentiators Competitors Won't Have

#	Differentiator	Why competitors miss it
1	**Click-to-verify Provenance Viewer** (bi-directional: output ⇄ source bbox)	They output Markdown only
2	**Self-verifying tables** — column/row totals are recomputed; mismatches	They stop at table structure

#	Differentiator	Why competitors miss it
	flagged and auto-re-extracted ("self-heal")	
3	**Chart re-plot verification** — extracted series is re-drawn over the original chart to prove accuracy	They output "[Figure 1]" or a caption
4	**Cost-aware escalation ladder** with live $/1k-pages meter	They call one big VLM for everything (expensive) or one OCR (inaccurate)
5	**Dual-path Office parsing**: native XML for fidelity + LibreOffice-rendered PDF to *recover bounding boxes* for DOCX/PPTX	They give DOCX no provenance
6	**Email with recursive attachments** (EML/MSG → body + each attachment parsed as child docs)	They reject email formats (explicitly named in the brief)
7	**Cross-page table stitching** with repeated-header de-duplication and visual link across pages	Tables break at page boundaries
8	**Benchmark Arena**: run our parser vs. naive baselines live, with real metrics (TEDS, edit distance, reading-order, IoU)	They claim accuracy without measuring
9	**License-clean, self-hostable stack** (permissive licenses only; audited in docs)	Many use AGPL/GPL libs that kill commercial self-hosting
10	**Review queue** — flagged blocks go to a human; corrections stored as future training/eval data	They have no human-in-the-loop
11	**MCP server + Python SDK** — plug into any AI assistant in 3 lines	They stop at a demo UI

> **License note (important for "fully self-hosted commercial viability"):** PyMuPDF is AGPL — avoid it in the core path. Prefer `pypdfium2` (Apache/BSD), `pdfminer.six` / `pdfplumber` (MIT), Docling (MIT),

> RapidOCR (Apache-2.0), python-docx/python-pptx/openpyxl (MIT-family). **Verify every license yourself before final docs.**

---

4. AI Features That Create the WOW Factor

1. **Chart → Data**: bar/line/pie/scatter detected, axes calibrated (pixel→value regression from OCR'd tick labels), series extracted, plus VLM cross-check. Output: a table + JSON series + "re-plot overlay."
2. **Equation → LaTeX**: detected equation regions decoded to LaTeX and rendered live with KaTeX next to the source crop.
3. **Self-Heal Loop**: sum-check fails → locate suspect cells (lowest OCR confidence in that row/column) → re-extract only those cells with the VLM → re-check. Show the "✗ → ✓" animation.
4. **Confidence Heatmap**: toggle overlays on the page: green/amber/red per block.
5. **"Refuses to Guess" mode**: blurry/occluded region → block marked `needs_review`, text withheld, crop shown. Judges *love* seeing an AI say "I'm not sure."
6. **Ask-the-Document with bbox citations**: answers cite block IDs; clicking a citation jumps to and highlights the source region.
7. **Talk to your document** (sponsor tie-in, optional): ElevenLabs TTS reads the answer aloud. *(Check current free-tier limits.)*
8. **Smart Router Explainer**: hover any region to see *why* it was routed (e.g., "image-only page → OCR; table detected → TableFormer; confidence 0.91").
9. **Semantic hierarchy**: heading levels, nested lists, caption↔figure linking, footnote↔reference linking.

---

5. Complete Modern Tech Stack

Backend (Python 3.11)

Layer	Choice	Notes
API	**FastAPI** + Uvicorn	async, auto OpenAPI docs
Job queue	**asyncio + ProcessPoolExecutor** (MVP) → **Redis + RQ/Arq** (advanced)	page-level parallelism
PDF rendering/text	**pypdfium2**, **pdfminer.six**, **pdfplumber**	permissive licenses, bbox from text layer
Layout analysis	**Docling** layout model (or DocLayNet-trained detector); heuristic XY-cut fallback	region typing
OCR	**RapidOCR (ONNX)**; Tesseract as fallback	word boxes + confidence
Table structure	pdfplumber (digital) · **Table Transformer / Docling TableFormer** (scanned)	
Math → LaTeX	**pix2tex** / **Texify** class model; VLM fallback	verify model licenses
Charts	**OpenCV** (axes, bars, lines, color segmentation) + VLM	
VLM gateway	**Gemini (free tier)** adapter + **Ollama (Qwen2.5-VL)** adapter	JSON-schema outputs, cache, budget guard
Office	**python-docx, python-pptx, openpyxl**, **LibreOffice headless**	native + render
Email	stdlib `email` (EML), `olefile` /extractor for MSG	recursive
Images	Pillow, OpenCV	deskew, denoise
Validation	**Pydantic v2**	canonical schema

Layer	Choice	Notes
DB	**SQLite** (MVP) → **Postgres (Supabase free)**	
Storage	local FS → **MinIO/S3-compatible**	page PNGs, crops
Eval	custom harness + `rapidfuzz`, `apted` (TEDS)	

Frontend

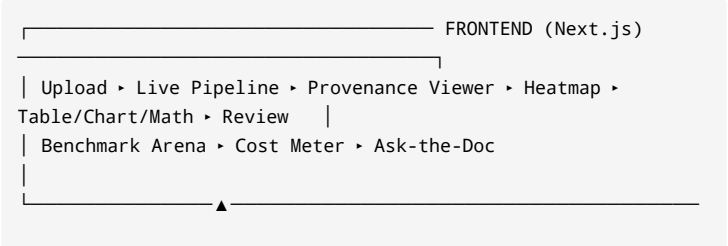
Layer	Choice
Framework	**Next.js (App Router) + TypeScript**
Styling	**Tailwind CSS + shadcn/ui** + Framer Motion
PDF viewer	**pdf.js (react-pdf)** + custom canvas bbox overlay layer
Charts	**Recharts** (re-plot overlay), **KaTeX** (equations)
State/data	TanStack Query + Zustand; SSE for live progress
Icons/Fonts	lucide-react, Inter + JetBrains Mono

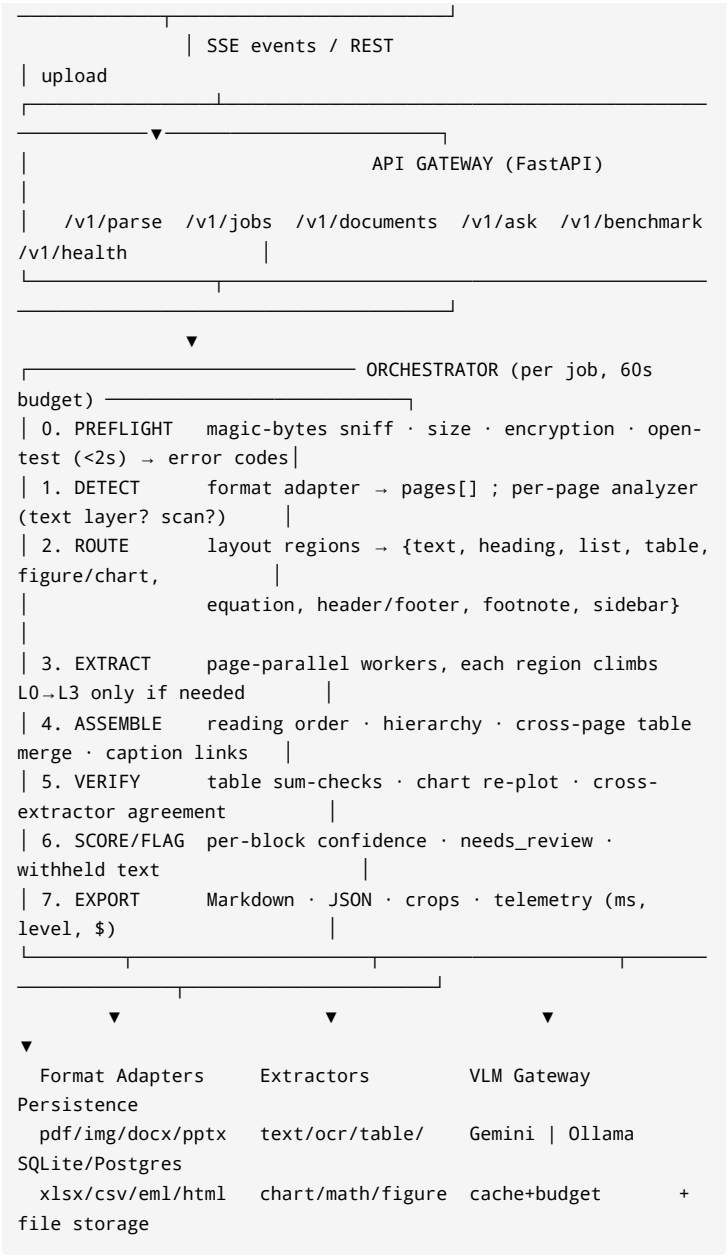
DevOps

Docker + docker-compose · GitHub Actions (lint + tests) · Vercel (frontend) · Hugging Face Spaces / Render / Fly (backend) · Cloudflare Tunnel (live-demo fallback from a laptop)

---

6. Full System Architecture




SSE events / REST
upload
API GATEWAY (FastAPI)
/v1/parse /v1/jobs /v1/documents /v1/ask /v1/benchmark /v1/health
ORCHESTRATOR (per job, 60s budget)
0. PREFLIGHT magic-bytes sniff · size · encryption · open-test (<2s) → error codes
1. DETECT format adapter → pages[] ; per-page analyzer (text layer? scan?)
2. ROUTE layout regions → {text, heading, list, table, figure/chart,
equation, header/footer, footnote, sidebar}
3. EXTRACT page-parallel workers, each region climbs L0→L3 only if needed
4. ASSEMBLE reading order · hierarchy · cross-page table merge · caption links
5. VERIFY table sum-checks · chart re-plot · cross-extractor agreement
6. SCORE/FLAG per-block confidence · needs_review · withheld text
7. EXPORT Markdown · JSON · crops · telemetry (ms, level, $)
Format Adapters
pdf/img/docx/pptx
xlsx/csv/eml/html
Extractors
text/ocr/table/
chart/math/figure
VLM Gateway
Gemini | Ollama
cache+budget
Persistence
SQLite/Postgres
+ file storage

6.1 Page & Region Detection Logic

- **Text-layer coverage** = chars on page / expected chars for area. `< threshold` **or** one image covering >70% of the page ⇒ **scanned page** ⇒ OCR path.
- **Hybrid pages** (text layer + embedded images): digital text is kept; images go through figure/chart/OCR routing.
- **Region typing** by layout model; heuristics fill gaps (ruled lines ⇒ lattice table; aligned x-gaps ⇒ stream table; math font/glyph density ⇒ equation).

6.2 Reading-Order Engine

1. Detect running **headers/footers** by repetition (same text/position on ≥50% of pages) ⇒ tagged `header` / `footer`, excluded from body flow.
2. Detect **columns** via x-range clustering + vertical whitespace gutters; recursive XY-cut.
3. **Footnotes**: bottom-of-page region with smaller font; linked to markers.
4. **Sidebars/callouts**: narrow, offset blocks ⇒ emitted after the main column they sit beside, flagged `sidebar`.
5. Global order assigned across pages; `reading_order` is an integer in JSON.

6.3 Cross-Page Table Stitching

Merge table A (page *n*) and B (page *n+1*) when: A ends within ~8% of bottom margin, B starts within ~8% of top margin, same column count, column x-positions within ±3%, and B's first row is either a repeated header (de-duplicated) or a continuation. Result carries `spans_pages: [n, n+1]` and **one bbox per page**.

6.4 Financial Integrity Checker (Verify stage)

- Normalize numbers: `(1,234)` ⇒ −1234, `$`, `%`, `₹`, `K/M/B`, units from header.
- Detect "Total / Subtotal / Net" rows/columns.
- Recompute sums; tolerance by rounding. Output `checks[]` with `ok/expected/found`.

- Failure ⇒ confidence ↓, suspect cells = lowest-confidence cells in that row/col ⇒ **Self-Heal** (VLM re-read of just those crops) ⇒ re-check.

6.5 Chart Digitizer

1. Classify chart type (CV heuristics + VLM).
2. OCR axis ticks/labels/legend/title.
3. Fit pixel→value mapping (linear/log) from ≥2 ticks.
4. Extract marks (bars by contour; lines by color path sampling; pie by angle).
5. Prefer **printed data labels** when present; otherwise derived values.
6. Cross-check CV result vs. VLM structured JSON; disagreement ⇒ confidence ↓.
7. **Re-plot overlay** to prove it.

6.6 Confidence Model

```
conf = clip( 0.45·extractor_conf
           + 0.25·cross_agreement      # two independent
extractors agree (e.g., text layer vs OCR sample)
           + 0.20·structural_checks    # sum-check, column-
consistency, axis-fit R²
           + 0.10·region_quality )     # blur/contrast/skew
prior

conf ≥ 0.80  → ok
0.50–0.80    → ok + "low_confidence" flag (amber)
< 0.50       → "needs_review" (red), text withheld if < 0.30,
crop attached
```

Calibrate thresholds against the gold set; report a reliability plot (confidence vs. actual accuracy) in the Arena — a rare, credible touch.

6.7 Fail-Safe (<60 s structured errors)

- **Preflight (<2 s):** magic bytes, extension mismatch, encrypted, zero-byte, truncated/corrupt, oversize.
- **Global deadline 55 s:** at deadline, return **partial result** with `status: "partial"` and per-page completion map.
- Error shape (always the same):

```
{
  "status": "error",
  "error": {
    "code": "E_CORRUPT_FILE",
    "message": "PDF xref table is damaged; could not open
document.",
    "stage": "preflight",
    "recoverable": false,
    "elapsed_ms": 412
  }
}
```

- Codes: `E_UNSUPPORTED_FORMAT`, `E_CORRUPT_FILE`, `E_ENCRYPTED`, `E_EMPTY_DOCUMENT`, `E_TOO_LARGE`, `E_TIMEOUT_PARTIAL`, `E_OCR_UNAVAILABLE`, `E_VLM_UNAVAILABLE` (degrades gracefully), `E_INTERNAL`.

6.8 Format Coverage Matrix

Format	Path	Provenance
PDF (digital/scanned/hybrid)	native + OCR	page + bbox
PNG/JPG/TIFF (multi-page)	OCR + layout	page + bbox
DOCX / PPTX	native XML **+** LibreOffice-rendered PDF aligned for bboxes	page + bbox (aligned)
DOC / PPT / XLS / RTF (legacy)	LibreOffice headless → modern format	as above
XLSX / CSV / TSV	native cell grid, merged cells, multiple sheets	sheet + cell range (`A1:F20`)
EML / MSG	headers + body + **recursive attachments**	message part / child doc
HTML / TXT / MD	DOM / text	DOM path / line range

7. Database Schema (PostgreSQL; SQLite-compatible for MVP)

```
CREATE TABLE documents (
  id            UUID PRIMARY KEY,
  parent_id     UUID REFERENCES documents(id),      -- email
attachments
  filename      TEXT NOT NULL,
  mime_type     TEXT,
  sha256        CHAR(64) NOT NULL,
  size_bytes    BIGINT,
  page_count    INT,
  status        TEXT CHECK (status IN
('queued','running','done','partial','error')),
  error_code    TEXT,
  error_message TEXT,
  created_at    TIMESTAMPTZ DEFAULT now(),
  finished_at   TIMESTAMPTZ
);
CREATE INDEX ON documents(sha256);                    -- cache
hits

CREATE TABLE jobs (
  id            UUID PRIMARY KEY,
  document_id   UUID REFERENCES documents(id),
  stage         TEXT,                                 --
preflight|detect|extract|verify|export
  progress      REAL,
  started_at    TIMESTAMPTZ, finished_at TIMESTAMPTZ,
  elapsed_ms    INT,
  est_cost_usd  NUMERIC(10,5) DEFAULT 0
);

CREATE TABLE pages (
  id            UUID PRIMARY KEY,
  document_id   UUID REFERENCES documents(id),
  page_no       INT,
  width_pt      REAL, height_pt REAL,
  kind          TEXT,                                 --
digital|scanned|hybrid
  image_path    TEXT,
  skew_deg      REAL,
```

```
  quality_score REAL,
  UNIQUE(document_id, page_no)
);

CREATE TABLE blocks (
  id            UUID PRIMARY KEY,
  document_id   UUID REFERENCES documents(id),
  page_id       UUID REFERENCES pages(id),
  type          TEXT,        --
heading|paragraph|list_item|table|figure|chart|equation|captio
n|footnote|header|footer|sidebar|code
  level         INT,         -- heading level / list depth
  reading_order INT,
  bbox          REAL[4],     -- normalized [x0,y0,x1,y1] in
0..1, origin top-left
  text          TEXT,        -- NULL when withheld
  confidence    REAL,
  flags         TEXT[],      --
low_confidence|needs_review|withheld|ocr|merged_cross_page|hea
led
  extractor     TEXT,        --
native|rapidocr|tableformer|cv_chart|vlm|...
  ladder_level  SMALLINT,    -- 0..3
  parent_block  UUID REFERENCES blocks(id),
  crop_path     TEXT,
  meta          JSONB
);
CREATE INDEX ON blocks(document_id, reading_order);

CREATE TABLE tables (
  block_id      UUID PRIMARY KEY REFERENCES blocks(id),
  n_rows INT, n_cols INT,
  spans_pages   INT[],
  html TEXT, markdown TEXT, csv TEXT,
  checks        JSONB                                -- sum-
check results
);
CREATE TABLE table_cells (
  id UUID PRIMARY KEY, table_id UUID REFERENCES
tables(block_id),
  r INT, c INT, rowspan INT DEFAULT 1, colspan INT DEFAULT 1,
  is_header BOOL, text TEXT, numeric_value NUMERIC, unit TEXT,
  col_path TEXT[], row_path TEXT[],
  bbox REAL[4], page_no INT, confidence REAL
```

```
);

CREATE TABLE figures (
  block_id UUID PRIMARY KEY REFERENCES blocks(id),
  kind TEXT,               -- chart|diagram|photo|logo
  chart_type TEXT,         -- bar|line|pie|scatter|...
  caption_block UUID REFERENCES blocks(id),
  description TEXT,
  series JSONB,            -- [{name, points:[{x,y}]}]
  axes JSONB,              -- {x:{label,scale,fit_r2}, y:
{...}}
  replot_agreement REAL
);

CREATE TABLE equations (
  block_id UUID PRIMARY KEY REFERENCES blocks(id),
  latex TEXT, display BOOL, decoder TEXT
);

CREATE TABLE reviews (
  id UUID PRIMARY KEY, block_id UUID REFERENCES blocks(id),
  reviewer TEXT, original_text TEXT, corrected_text TEXT,
  created_at TIMESTAMPTZ DEFAULT now()
);

CREATE TABLE benchmark_runs (
  id UUID PRIMARY KEY, system TEXT,            --
parseanything|pypdf|pdfplumber|...
  corpus TEXT, created_at TIMESTAMPTZ DEFAULT now(),
  metrics JSONB                                -- {teds,
text_ned, order_ned, bbox_iou, chart_acc, p50_ms, p95_ms,
pages_per_s, usd_per_1k}
);
```

---

8. API Structure

Base: `/v1` · JSON · OpenAPI at `/docs`.

Method	Path	Purpose
POST	`/parse`	Upload file (multipart) →

Method	Path	Purpose
		`{job_id, document_id}`; `?sync=true` waits ≤55 s
GET	`/jobs/{id}`	Status, stage, progress, elapsed, est. cost
GET	`/jobs/{id}/events`	**SSE** stream of pipeline events (stage, page done, region routed)
GET	`/documents/{id}`	Full structured JSON
GET	`/documents/{id}/markdown`	Markdown with provenance anchors
GET	`/documents/{id}/pages/{n}/image`	Rendered page PNG
GET	`/documents/{id}/blocks/{block_id}`	One block (+ crop URL)
GET	`/documents/{id}/tables/{block_id}?format=csv|html|json`	Table export
GET	`/documents/{id}/flags`	All blocks needing review
POST	`/blocks/{block_id}/review`	Submit human correction
POST	`/documents/{id}/ask`	Q&A with block citations
POST	`/benchmark/run`	Run corpus vs. selected systems

Method	Path	Purpose
GET	`/benchmark/runs/{id}`	Metrics
GET	`/cost/estimate?pages=1000`	Cost model output
GET	`/health`	Models loaded, VLM reachability

Canonical JSON (abridged)


```
{
  "schema_version": "1.0",
  "document":
{"id":"…","filename":"credit_agreement.pdf","pages":42,"status
":"done",

"elapsed_ms":18430,"est_cost_usd":0.011,"flagged_blocks":3},
  "pages":
[{"page_no":1,"kind":"digital","width_pt":612,"height_pt":792}
],
  "blocks": [

{"id":"b_0007","type":"heading","level":2,"page":3,"reading_or
der":7,
     "bbox":[0.08,0.11,0.62,0.14],"text":"Debt
Schedule","confidence":0.97,
     "flags":[],"extractor":"native","ladder_level":0},
    {"id":"b_0008","type":"table","page":3,"spans_pages":
[3,4],"reading_order":8,
     "bbox":[0.08,0.16,0.92,0.88],"confidence":0.91,
     "table":{"n_rows":14,"n_cols":6,
       "header_rows":2,
       "cells":
[{"r":0,"c":1,"colspan":2,"text":"FY2024","is_header":true,"bb
ox":[…]}],
       "checks":
[{"type":"column_sum","col":3,"row":13,"expected":48210,"found
":48210,"ok":true}],
       "markdown":"…","html":"…","csv":"…"}},
    {"id":"b_0021","type":"chart","page":5,"bbox":
[…],"confidence":0.84,
     "figure":{"chart_type":"bar","series":
```

```
[{"name":"Revenue","points":[{"x":"Q1","y":120.4}]}],
               "replot_agreement":0.96}},
    {"id":"b_0030","type":"equation","page":6,"latex":"E =
mc^2","confidence":0.88},
    {"id":"b_0044","type":"paragraph","page":8,"text":null,
     "flags":
["needs_review","withheld"],"confidence":0.22,"crop":"/…/b_004
4.png"}
  ]
}
```


---

9. Frontend + Backend Plan

Backend modules

```
core/      schema.py  errors.py  config.py  telemetry.py
ingest/    sniff.py  preflight.py
adapters/{pdf,image,docx,pptx,xlsx,csv,email,html,legacy}.py
analyze/   page_analyzer.py  layout.py  router.py
extract/   native_text.py  ocr.py  tables_digital.py
tables_scanned.py
           figures.py  charts.py  equations.py
assemble/  reading_order.py  hierarchy.py  table_stitch.py
captions.py
verify/    numeric.py  integrity.py  selfheal.py  replot.py
score/     confidence.py  flagging.py
export/    markdown.py  json_export.py
vlm/       gateway.py  gemini.py  ollama.py  cache.py
budget.py
api/       main.py  routes_*.py  sse.py
eval/      metrics.py  runner.py  baselines.py  corpus/
```

Frontend pages

1. `/` — Landing + drag-drop + "**Try the Torture Test**" sample picker
2. `/doc/[id]` — **Provenance Viewer** (the star)
3. `/arena` — Benchmark Arena
4. `/review` — Flag review queue
5. `/docs` — API playground

10. Stunning UI/UX Strategy

Visual language: dark "forensic lab" theme — near-black canvas (`#0B0F14`), electric-cyan accent (`#22D3EE`), confidence palette (green `#22C55E` , amber `#F59E0B` , red `#EF4444`), glassy panels, subtle grid background, mono font for JSON.

Provenance Viewer layout (3 panes):

```
┌──────────────── top bar: file · status · time · $cost · [Heatmap] [Blocks] [Export ▾] ────────────────┐
│  PAGE THUMBS │            SOURCE PAGE (pdf.js + bbox overlays)      │  OUTPUT (tabs)          │
│  (confidence │  hover = tooltip: type · conf · extractor · level    │  Markdown | JSON | Tables│
│   dots)      │  click = syncs & scrolls output pane                 │  Charts | Math | Flags  │
└──────────────┴──────────────────────────────────────────────────────┴─────────────────────────┘
bottom drawer: Pipeline timeline (Detect → Route → Extract → Verify → Flag) with per-region ladder badges
```

Micro-interactions that sell it

- Live pipeline: page tiles flip from "queued" → "scanning" → "routed" with colored region boxes *drawing themselves* as SSE events arrive.
- Click a table cell → the exact pixel crop slides in beside it.
- Sum-check badge animates ✗ → "healing…" → ✓.
- Chart panel: slider fades between **original** and **re-plot** overlay.
- Cost meter: odometer-style `$0.0042` that ticks while parsing; ladder badge L0/L1/L2/L3 per region.
- "Refuses to guess" red hatched region with a lock icon and **Send to review**.

---

11. MVP Features (build these first — they alone beat most teams)

1. Single `/parse` endpoint + preflight + structured errors

2. PDF digital + scanned (OCR) with page detection
3. Typed blocks + bboxes + confidence in JSON, Markdown export
4. Multi-column reading order + header/footer removal
5. Digital table extraction with merged cells + multi-row headers + cross-page merge
6. Image, DOCX, PPTX, XLSX/CSV adapters
7. Provenance Viewer with bbox overlay + click-sync
8. Confidence heatmap + flagged-block list
9. Chart → data (bar + line) via VLM + OCR labels
10. Equation → LaTeX
11. Numeric sum-check on tables
12. Docker one-liner + README + 15-doc test corpus

---

12. Advanced Features (judge-impact layer)

- Self-heal loop + re-plot overlay
- Cost meter + ladder visualization
- Benchmark Arena with reliability plot
- Email (EML/MSG) with recursive attachments
- Legacy formats via LibreOffice
- Ask-the-Document with bbox citations (+ ElevenLabs voice answer)
- Review queue → corrections saved as eval data
- MCP server + `pip install` SDK (`parseanything.parse("file.pdf")`)
- Offline / air-gapped mode (Ollama VLM) — proves "fully self-hosted"
- Page-parallel worker pool + sha256 result cache

---

13. Free APIs / Tools / Services

Need	**Free option *(verify current limits)***
VLM	Gemini API free tier · Ollama + Qwen2.5-VL (local)
OCR	RapidOCR · Tesseract

Need	Free option *(verify current limits)*
Layout/Tables	Docling · Table Transformer (HF)
Math OCR	pix2tex · Texify-class models
TTS (bonus)	ElevenLabs free tier
DB	SQLite · Supabase free Postgres
Frontend hosting	Vercel Hobby
Backend hosting	Hugging Face Spaces (Docker, free CPU) · Render/Fly free tiers
Tunnel	Cloudflare Tunnel
CI	GitHub Actions
Datasets for eval	OmniDocBench, DocLayNet, PubTabNet/FinTabNet, olmOCR-bench subsets *(check licenses)* + our own gold set
Test docs	SEC EDGAR filings (public), arXiv papers, government PDFs

---

14. Deployment Strategy

1. **Primary:** Vercel (frontend) + Docker image on Hugging Face Spaces/Render (backend). Pre-download models in the image build to avoid cold-start downloads.
2. **Self-host:** `docker compose up` — brings API + worker + Redis + frontend + (optional) Ollama. One command, documented.
3. **Demo insurance:** full stack on the team laptop, exposed via Cloudflare Tunnel; **pre-parsed results cached** for the torture-test corpus so the demo works even if Wi-Fi dies.
4. **Config via env:** `VLM_PROVIDER=gemini|ollama|none`, `VLM_BUDGET_USD`, `MAX_PAGES`, `GLOBAL_DEADLINE_S=55`.
5. **Graceful degradation:** VLM unreachable ⇒ L0–L2 only, affected blocks flagged (never crash).

Cost model (show this slide — with *measured* numbers)

```
cost_per_1k_pages = (1000 × escalation_rate ×
avg_vlm_calls_per_page × cost_per_vlm_call) + infra_per_1k
```

- Fill in `escalation_rate` from the benchmark (% of pages with ≥1 L3 call), and `cost_per_vlm_call` from the provider's *current* pricing.
- Self-hosted Ollama path ⇒ `cost_per_vlm_call ≈ 0` (compute only).
- Target: **well under $10/1k pages**; present as a range with assumptions, not a single magic number.

---

15. 24-Hour Hackathon Execution Roadmap (team of 4)

Roles: A Core pipeline/PDF/OCR/reading-order · **B** Tables/Charts/Math/VLM · **C** Backend API/infra/formats/benchmark · **D** Frontend/UX/pitch

Hour	A	B	C	D
0–1	Repo, schema, env	Model downloads, VLM gateway stub	Docker, FastAPI skeleton, preflight	Next.js shell, theme, upload
1–4	Page analyzer, native text+bboxes, OCR	Digital tables (pdfplumber) + spans	Orchestrator, job/SSE, SQLite	Viewer: pdf.js + bbox overlay
4–8	Layout/region router, reading order	Multi-row headers, cross-page stitch	Office/image/xlsx adapters	Output panes (MD/JSON/Tables)
8–12	Hierarchy, header/footer,	Chart digitizer v1,	Markdown/JSON export, error	Heatmap, flags panel, live pipeline

Hour	A	B	C	D
	confidence	equations	codes, tests	
12	**Checkpoint: end-to-end MVP on 5 docs. Freeze MVP scope.**			
12–16	Perf (process pool), caching	Sum-check + self-heal, re-plot	Email adapter, legacy convert	Chart/math viewers, cost meter
16–19	Gold set + torture set	Metrics harness (TEDS etc.)	Benchmark baselines + Arena API	Arena page, Ask-the-Doc UI
19–21	Bugfix on corpus	Tune thresholds, calibration	Deploy (Vercel + Spaces), README	Polish, animations
21–23	Demo rehearsal ×3, backup video, PPT			
23–24	**Code freeze at H22**; final deploy check; sleep 30 min if possible			

Cut-line if behind (drop in this order): voice/TTS → MCP/SDK → Ask-the-Doc → email attachments → legacy formats → scanned-table model (use VLM) → self-heal. *Never cut:* Provenance Viewer, confidence/flags, fail-safe errors, cross-page tables, Benchmark numbers.

---

16. Judge-Focused Demo Strategy (5 minutes)

Prep: a "Torture Test" folder with 6 files: (1) scanned debt schedule (photo, slightly skewed), (2) two-column research paper with equations, (3) multi-page financial table with repeated headers, (4) PPTX with a bar chart, (5) EML with a PDF attachment, (6) a deliberately corrupted PDF.

Time	Beat	What judges see
0:00	Hook	"A missing line item in a debt schedule can cost millions. Our parser can prove it didn't miss one."
0:20	Drop all 6 files at once	One entry point; live pipeline; routing badges per file
1:00	Scanned debt schedule	Heatmap on; click a total → source crop; **sum-check ✓**
1:40	Self-heal	Show a pre-seeded OCR error (✗) → healing → ✓
2:10	Two-column paper	Reading order numbers overlay; equation → LaTeX rendered
2:40	PPTX chart	Chart → data table; slide original ↔ re-plot slider
3:10	Cross-page table	Linked across pages, repeated header de-duplicated
3:30	Corrupt file	Structured error in ~1 s — "never silent, never hallucinated"
3:50	**Benchmark Arena**	Us vs. baseline: TEDS, reading-order, chart accuracy, pages/s, $/1k
4:30	Cost meter + self-host	"88% of regions never left the free tier" *(use your measured number)*; `docker compose up`
4:50	Close	The one-liner

Rules: pre-cache results; always have the recorded video ready; narrate *why* each routing decision happened; show one honest limitation (judges trust that).

---

17. 2-Minute Winning Pitch

> **[0:00]** Imagine an AI analyst reading a 200-page credit agreement. It misses one line in a debt schedule. A deal is mispriced. Nobody can tell *why*, because the parser never showed its work.
>
> **[0:20]** That's the problem Cenizas Labs and every vertical AI company faces: parsers that flatten tables, drop charts, scramble columns — and never say *"I'm not sure."*
>
> **[0:35]** We built **ParseAnything** — one entry point for PDFs, scans, Word, PowerPoint, Excel, images, even emails with attachments. It **detects** what each region is, **routes** it to the right specialist, **extracts** typed blocks with bounding boxes, and then does what no baseline does: it **verifies itself**.
>
> **[1:00]** Tables recompute their own totals. Charts are re-plotted over the original to prove the numbers. Anything uncertain is **flagged — never guessed.** Every block links back to the exact pixels it came from.
>
> **[1:20]** It's production-ready: a cost-aware ladder keeps most regions on free CPU and calls a vision model only when needed — so we land far below $10 per thousand pages. It's license-clean, self-hosted with one Docker command, and fails with structured errors in under a minute.
>
> **[1:45]** Here are our measured results against standard baselines. *(Arena slide.)*
>
> **[1:55]** ParseAnything: **every page parsed, every number traceable.** Thank you.

---

18. Likely Judge Questions & Winning Answers

1. **How is this better than just sending every page to a VLM?**
   Cost, latency, and hallucination risk. The ladder uses deterministic extraction first; the VLM is a *verifier and fallback*, not the foundation. Native text is exact; a VLM can invent digits.
2. **How do you guarantee you don't hallucinate?**
   We don't *guarantee*; we *measure and flag*. Text from native layers is exact; OCR/VLM outputs get confidence from agreement and structural checks; below threshold the text is withheld and the block is routed to review.

3. **How did you get bounding boxes for DOCX/PPTX?**
   Dual path: parse native XML for structure, render via LibreOffice to PDF, then align blocks by text matching to recover page + bbox. Spreadsheets use sheet + cell-range provenance.

4. **What are your accuracy numbers?**
   (Use the Arena.) TEDS for tables, normalized edit distance for text, reading-order NED, bbox IoU, chart-value accuracy — on our gold set and a public-benchmark subset, with baselines side by side. We show the limitations too.

5. **What's your cost per 1,000 pages?**
   Formula-driven and measured: escalation rate × VLM calls × current price + infra. Self-hosted VLM mode drives marginal API cost to ~$0. Live meter in the UI.

6. **Can it be fully self-hosted / commercially licensed?**
   Yes: permissive-licensed core, Docker compose, optional local VLM via Ollama. We avoided AGPL/GPL libraries in the core path (audit in `docs/LICENSES.md`).

7. **What about throughput?**
   Page-level process-pool parallelism, sha256 cache, deterministic L0 path at millisecond scale. We report p50/p95 and pages/sec from the harness.

8. **How do you handle scanned, skewed, or low-quality pages?**
   Quality scoring → deskew/denoise → OCR with word-level confidence → if still low, escalate or flag. Blurry regions are *withheld*, not guessed.

9. **How do you merge tables across pages?**
   Geometric + structural matching (position, column count/x-alignment, repeated headers), carrying per-page bboxes; confidence reduced if ambiguous.

10. **What about handwritten text or non-English?**
    Honest answer: out of MVP scope for accuracy claims; OCR language packs are configurable and the VLM path can help; we flag low-confidence output. Roadmap item.

11. **How do charts get accurate numbers?**
    Printed labels first; otherwise axis calibration from tick OCR and CV mark

extraction; VLM cross-check; re-plot agreement score exposed per chart.

12. **What happens when the VLM is down or rate-limited?**
    Graceful degradation: L0–L2 continue; affected regions flagged; `E_VLM_UNAVAILABLE` reported in metadata; no crash.
13. **Who is the customer and what's the business model?**
    Vertical AI builders (PE, IB, legal diligence, compliance). Usage-based API ($/1k pages), enterprise self-hosted license, and a review-queue/audit add-on.
14. **What's your moat?**
    Verification + provenance + the corrections dataset from the review queue — it compounds into better thresholds and eval sets.
15. **What would you build next?**
    Handwriting, form-field extraction, layout-model fine-tuning on corrected data, redaction/PII, streaming for 1000+ page files.

---

19. Professional PPT Structure (12 slides)

1. **Title** — ParseAnything · tagline · team
2. **The Cost of a Missed Line** — high-stakes domain story (Cenizas Labs context)
3. **Why Current Parsers Fail** — the 5 failure modes from the brief (visual icons)
4. **Our Insight** — "A parser that knows what it doesn't know"
5. **Solution Overview** — Detect → Route → Extract → **Verify** → Flag
6. **Architecture** — diagram + escalation ladder
7. **Live Demo** (switch to product) — or embedded video
8. **Technical Depth** — reading order, table stitching, chart digitizer, integrity checker
9. **Benchmark Results** — Arena screenshot, metrics vs. baselines, reliability plot
10. **Production Readiness** — cost/1k pages, throughput, self-host, license audit, failure handling
11. **Business & Roadmap** — customers, pricing, moat, next steps

12. **Close** — "Every page parsed. Every number traceable." + QR to repo/demo

---

20. Technical Documentation Content

docs/ARCHITECTURE.md — pipeline stages, ladder, data flow diagram, module responsibilities.
docs/SCHEMA.md — canonical JSON schema (also published as JSON Schema file), field semantics, bbox coordinate convention (normalized, top-left origin).
docs/API.md — endpoints, examples (`curl`, Python), error codes table.
docs/ROUTING.md — page/region classification rules, thresholds, escalation policy.
docs/CONFIDENCE.md — scoring formula, calibration method, flagging policy.
docs/TABLES.md — spans, header paths, cross-page stitching, integrity checks.
docs/CHARTS_MATH.md — chart digitizer & equation decoding method and limits.
docs/BENCHMARKS.md — corpus description, metrics definitions (TEDS, NED, IoU), baselines, results, limitations.
docs/COST_PERF.md — cost model, measured escalation rate, throughput numbers, hardware used.
docs/SELF_HOSTING.md — Docker, env vars, offline mode, model download.
docs/LICENSES.md — dependency license audit.
docs/LIMITATIONS.md — honest list (handwriting, rotated tables, etc.).

---

21. README.md Structure

```
# ParseAnything
<tagline + badges: build, license, docker>
<GIF: Provenance Viewer clicking a number → source crop>

## Why
## Features (table)
## Quick Start
```

```
  - docker compose up
  - open http://localhost:3000
  - curl example
## Supported Formats
## Output Format (JSON excerpt + Markdown excerpt)
## How It Works (pipeline diagram + escalation ladder)
## Benchmarks (table + link to docs/BENCHMARKS.md)
## Cost & Performance
## Configuration (.env table)
## API Reference (link)
## SDK & MCP usage
## Project Structure
## Limitations & Roadmap
## License & Third-Party Licenses
## Team
```

---

22. Resume-Ready Project Description

ParseAnything — High-Fidelity Universal Document Ingestion Engine
(DataQuest 3.0, VIT Chennai)

- Built a provenance-first document parser converting PDFs, scans, DOCX/PPTX/XLSX, images and emails into typed Markdown + JSON with bounding boxes and per-block confidence scores.
- Designed a cost-aware 4-level extraction ladder (native → CPU OCR/CV → specialist models → vision-LLM) that limits paid model calls to low-confidence regions, targeting <$10 per 1,000 pages with a fully self-hostable, permissively-licensed stack.
- Implemented multi-column reading-order recovery, cross-page table stitching, chart-to-data digitization with re-plot verification, and equation-to-LaTeX decoding.
- Added a financial integrity checker with a self-healing re-extraction loop; evaluated with TEDS, edit-distance, reading-order and bbox-IoU metrics against baselines.
- Shipped a FastAPI + Next.js platform with an interactive provenance viewer, benchmark arena, review queue, Docker deployment and structured <60 s failure handling.

(Replace bracketed targets with your measured numbers before submitting.)

23. GitHub-Ready Folder Structure

```
parseanything/
├── README.md
├── LICENSE
├── docker-compose.yml
├── .env.example
├── .github/workflows/ci.yml
├── backend/
│   ├── pyproject.toml
│   ├── Dockerfile
│   ├── app/
│   │   ├── core/        (schema.py, errors.py, config.py,
telemetry.py)
│   │   ├── ingest/      (sniff.py, preflight.py, adapters/…)
│   │   ├── analyze/     (page_analyzer.py, layout.py,
router.py)
│   │   ├── extract/     (native_text.py, ocr.py,
tables_digital.py, tables_scanned.py,
│   │   │                 figures.py, charts.py, equations.py)
│   │   ├── assemble/    (reading_order.py, hierarchy.py,
table_stitch.py, captions.py)
│   │   ├── verify/      (numeric.py, integrity.py,
selfheal.py, replot.py)
│   │   ├── score/       (confidence.py, flagging.py)
│   │   ├── export/      (markdown.py, json_export.py)
│   │   ├── vlm/         (gateway.py, gemini.py, ollama.py,
cache.py, budget.py)
│   │   ├── api/         (main.py, routes_parse.py,
routes_docs.py, routes_ask.py,
│   │   │                 routes_bench.py, sse.py)
│   │   └── db/          (models.py, session.py)
│   ├── eval/            (metrics.py, runner.py, baselines.py,
corpus/, gold/)
│   └── tests/           (unit/, e2e/, failure_modes/)
├── frontend/
│   ├── package.json
│   ├── src/app/         (page.tsx, doc/[id]/page.tsx,
arena/page.tsx, review/page.tsx)
│   ├── src/components/  (Viewer/, BBoxLayer.tsx,
HeatmapToggle.tsx, TableView.tsx,
│   │                     ChartView.tsx, MathView.tsx,
```

```
CostMeter.tsx, PipelineTimeline.tsx,
│   │                    AskPanel.tsx, ui/)
│   └── src/lib/         (api.ts, sse.ts, store.ts)
├── sdk/                 (python/parseanything/, mcp_server/)
├── samples/             (torture_test/)
├── docs/                (ARCHITECTURE.md, SCHEMA.md, API.md,
BENCHMARKS.md, …)
└── scripts/             (download_models.sh,
build_gold_set.py, seed_demo.py)
```

---

24. Evaluation Methodology (so your numbers are credible)

Capability	Metric	Baseline to beat
Text fidelity	Normalized edit distance (NED) / CER	`pypdf` / `pdfplumber` plain text
Reading order	NED over block sequence	naive top-to-bottom, left-to-right sort
Tables	**TEDS** / TEDS-struct, cell-level F1	pdfplumber default
Charts	Mean relative error of extracted values; series-count accuracy	"figure placeholder" (0)
Math	LaTeX edit distance / exact-match after normalization	dropped/garbled text
Provenance	Bbox IoU ≥ 0.5 / 0.75 against gold boxes	none (baselines have no bboxes)
Confidence	Reliability plot, ECE	—
Robustness	% of corrupt/unsupported files returning structured error <60 s	crash/timeouts
Throughput	pages/sec, p50/p95 latency	—
Cost	$/1k pages (measured escalation rate)	all-VLM approach

Gold set: ~30 documents (SEC filings, arXiv papers, scans, Office files) hand-annotated lightly (tables + a few blocks per doc is enough). Add an OmniDocBench subset if time allows. **Report limitations honestly.**

A COMPLETE WINNING EXECUTION BLUEPRINT

Idea

ParseAnything = *Detect → Route → Extract → Verify → Flag*, with a click-to-verify Provenance Viewer. A parser that proves its work and admits uncertainty.

Architecture

FastAPI orchestrator with a 55-second global deadline → format adapters → page analyzer/router → page-parallel extractors on a 4-level cost ladder → assembler (reading order, cross-page tables) → verifier (sum-checks, chart re-plot, self-heal) → confidence scorer → Markdown/JSON + crops. Next.js viewer over SSE.

Coding (order of attack)

1. Schema + preflight + errors (Hour 0–1)
2. PDF native + OCR + bboxes → JSON (1–4)
3. Viewer with bbox overlay (parallel, 1–4)
4. Tables + reading order + cross-page stitch (4–8)
5. Other formats (4–8)
6. Confidence/flags/heatmap (8–12)
7. **MVP checkpoint at Hour 12**
8. Charts, math, sum-check, self-heal, re-plot (12–16)
9. Benchmark harness + Arena (16–19)
10. Polish, deploy, docs (19–21)

Deployment

Vercel + Dockerized backend on HF Spaces/Render; `docker compose up` for self-host; laptop + Cloudflare Tunnel as live backup; cached demo results.

Presentation

12-slide deck, 5-minute live demo on a prepared torture-test corpus, 2-minute pitch ending on measured benchmark results, recorded backup video.

Judging Strategy

- **Lead with trust**, not features: "every number traceable, nothing hallucinated."
- **Show numbers**: Arena metrics + cost-per-1k with stated assumptions.
- **Show failure on purpose**: corrupt file → clean error; blurry region → withheld.
- **Pre-empt the hard questions** (cost, licensing, hallucination, scale) with slides and the Q&A above.
- **Be honest about limits** — it is the strongest credibility signal in a room full of overclaims.

> **Final reminder:** every number in this document marked as a target must be replaced by a number you actually measured before you present it.